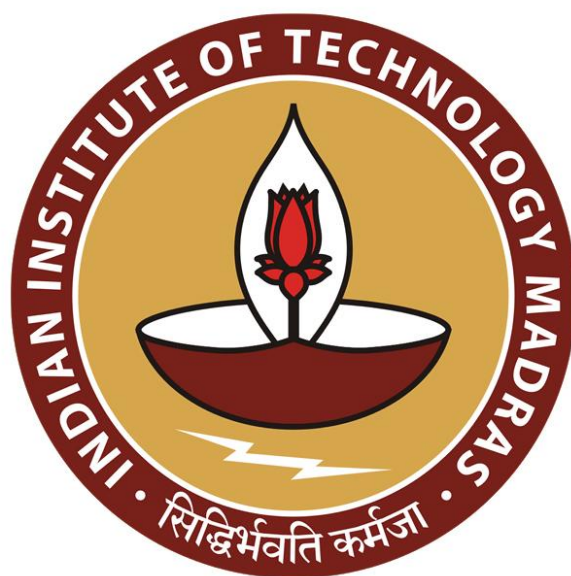
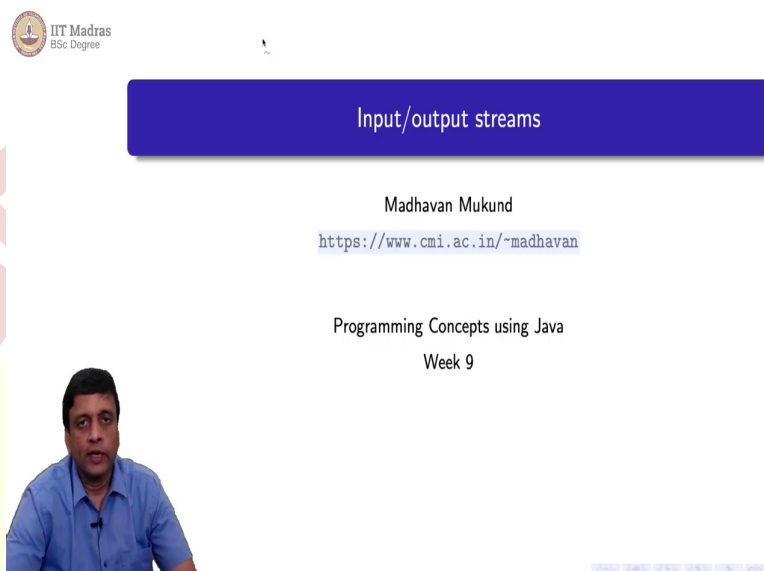




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Input/Output Streams

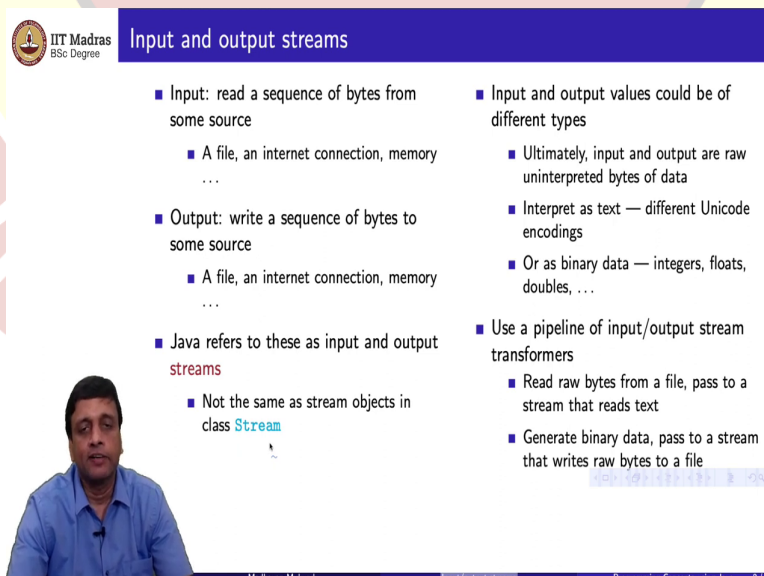
(Refer Slide Time: 00:13)



The slide features a blue header with the text "Input/output streams". Below the header, the name "Madhavan Mukund" and the URL "https://www.cmi.ac.in/~madhavan" are displayed. The main content area shows "Programming Concepts using Java" and "Week 9". A video feed of the professor is visible in the bottom left corner. The IIT Madras logo is in the top left corner.

So, let us look at how to do input and output in Java.

(Refer Slide Time: 00:16)



The slide features a blue header with the text "Input and output streams". The main content area contains a list of bullet points explaining input and output streams. A video feed of the professor is visible in the bottom left corner. The IIT Madras logo is in the top left corner.

- Input: read a sequence of bytes from some source
 - A file, an internet connection, memory ...
- Output: write a sequence of bytes to some source
 - A file, an internet connection, memory ...
- Java refers to these as input and output **streams**
 - Not the same as stream objects in class `Stream`
- Input and output values could be of different types
 - Ultimately, input and output are raw uninterpreted bytes of data
 - Interpret as text — different Unicode encodings
 - Or as binary data — integers, floats, doubles, ...
- Use a pipeline of input/output stream transformers
 - Read raw bytes from a file, pass to a stream that reads text
 - Generate binary data, pass to a stream that writes raw bytes to a file

So in general, input requires us to read a sequence of bytes from some external source. So, this could be a file, it could be the keyboard, what is called the console. It could even be in

these days, an internet connection, you might be reading a stream of data from an external website.

Or you might just want to be reading data from some block of memory, think of it as an abstract file and read from memory. So, all of these conceptually are the same thing, you have something which is outside your program, which is feeding you a sequence of data and the, in a raw form, the sequence of data is a sequence of bytes.

And symmetrically, you might want to write, write, so you might want to send out data to a file or to an internet connection or to a block of memory, or on to just the screen, for example. So, Java refers to this process of reading and writing sequences of bytes as input and output streams.

Now, this is a somewhat perhaps awkward use of terminology, because we have just been studying a very specific type of streams, which is this class stream, which is a kind of sequence of values, which we can then process by passing it through a number of filtering transformations.

So, even though there is a similarity, you can think of a sequence of data coming in and out as a stream of data. But technically, the word stream in the context of input and output streams is not quite the same as the word stream in the sense of the previous discussion on Java stream class.

So, one of the big problems with input and output is that we have to interpret this data, so in a raw form, all data is written in the form of binary zeros and ones. And usually packaged in bytes, which are 8 bits, 8 binary digits. But whether these 8 binary digits, or groups of eight by like 2 bytes, or 4 bytes, or 8 bytes together constitute a character do they constitute an integer is it to be written thought of as a floating point number. So, we have to do this transformation, to decide whether to interpret these big sequences, these bytes that we read in these input output streams as meaningful data.

So, one way to do this would be to have a different operation for each type. So, supposing you want to read data from a text file, then you have a function, which reads text information from a file, you want to write binary data to a network connection, then you have a separate function with writes binary data, only two network connections. So, this would mean that for every combination of input and output source, and every type of data that you want to read

and every other transformation that you want to do, you would have a different way of invoking the function.

So, Java has a slightly different approach to this he tries to modularize this process into the different components. So at the raw level, you would have something that can read and write bytes. Now, you can read and write bytes from different external sources. So, there is a separate component, which allows you to separate IO stream as it is, which allows you to connect an external device like a file or a network connection, as a stream from which you can read bytes, then you might want to transform these bytes and read them as text read them as binary data and so on.

So, by building this kind of a pipeline of these IO streams, you can build a transformation which takes a source converts the bytes into a meaningful form and processes it and in the reverse direction, takes a meaningful data converts it back to bytes and writes it.

(Refer Slide Time: 03:55)

IIT Madras
BSc Degree

Reading and writing raw bytes

- Classes `InputStream` and `OutputStream`
- Read one or more bytes — abstract methods are implemented by subclasses of `InputStream`
- Check availability before reading

```
abstract int read();
int read(byte[] b);
byte[] readAllBytes();
// ... and more

InputStream in = ...;
int bytesAvailable = in.available();
if (bytesAvailable > 0)
{
    var data = new byte[bytesAvailable];
    in.read(data);
}
```

Madhavan Makund | Input/output streams | Programming Concepts using Java | 3/8

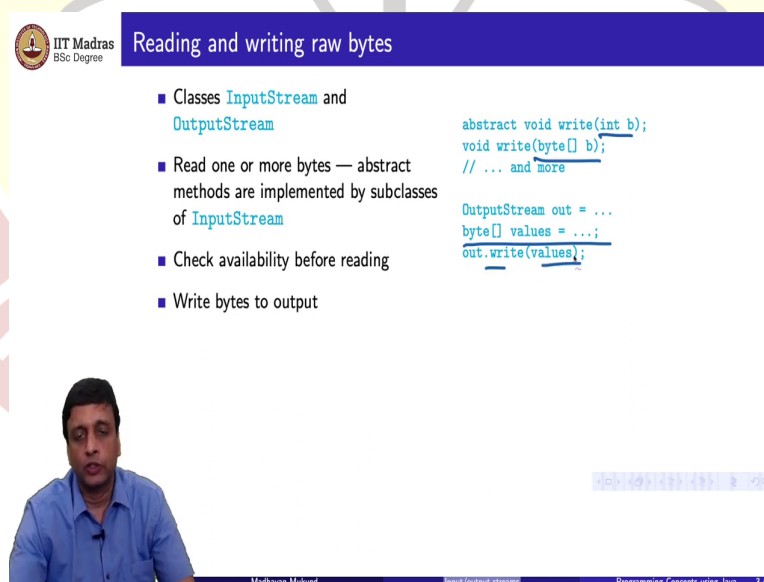
So, at the raw level, we read and write bytes. So, these are described in the Java classes input stream and output stream. So, if you look at input stream, it gives you some methods to read bytes. So, the most abstract of these which is actually not implemented directly by input stream, but is concretized by subclasses, is called read. Now you notice that it reads a byte but it actually returns an int. And the idea is that usually signifies a minus 1 if there is a terminating condition. So, that is why an int.

Now, you could have more general generous reads like you could read into an array of bytes. And this will return to you the number of bytes that were read or you could read the entire input stream as bytes into an array of bytes. So, there are many more these are just a few examples of the kind of input read functions which are available in `InputStream`.

So for example, when you try to read and something is not available, then you have to wait. So this is technically called blocking. So, the program will wait or it will block until the data is available. So, if you want to be sure that you are reading data that is available, you might want to check in advance. So, there is a function that input stream class gives you a call available.

So, if you have an input stream, then you can invoke `available` on that input stream, so on in and it will tell you how many bytes are currently available to read. And then you could read use this form of the function which reads into a byte array. By checking if you have a non-zero `available`, non-zero number of bytes available, then you can read into a byte array of that size. So, this is a way of combining the different components of input stream in order to read.

(Refer Slide Time: 05:52)



Reading and writing raw bytes

- Classes `InputStream` and `OutputStream`
- Read one or more bytes — abstract methods are implemented by subclasses of `InputStream`
- Check availability before reading
- Write bytes to output

```
abstract void write(int b);  
void write(byte[] b);  
// ... and more  
  
OutputStream out = ...  
byte[] values = ...;  
out.write(values);
```

Madhavan Mukund | Input/output streams | Programming Concepts using Java | 3 / 8

So symmetrically, you have an output stream class, which gives you these abstract and concrete ways to write data. So, you can write a single byte, or you can write an array of bytes and so on. So, for example, here is a situation where you have an array of bytes and you write it out in one shot.

(Refer Slide Time: 06:10)



IIT Madras
BSc Degree

Reading and writing raw bytes

- Classes `InputStream` and `OutputStream`
- Read one or more bytes — abstract methods are implemented by subclasses of `InputStream`
- Check availability before reading
- Write bytes to output
- Close a stream when done — release resources
- Flush an output stream — output is buffered

```
abstract void write(int b);  
void write(byte[] b);  
// ... and more  
  
OutputStream out = ...  
byte[] values = ...;  
out.write(values);
```

```
in.close();
```

```
out.flush();
```



Madhavan Mukund

Input/output streams

Programming Concepts using Java

3 / 8

So, one of the things that has to be done when you are dealing with streams is you are really connecting at the, at some level an external resource to your program. And so, underlying this there is some mechanism which has to be invoked in order to do this. So, there are some other things which have to be done to make these streams connect to each other and connect to the file or the network or something.

So, in order to signal to the system that this connection is no longer needed, you should always close a stream once you are done with it. So, close is a function which is associated with streams, which returns the resource that you are using to do your input and output. The other thing that happens normally with output in particular, is that the output is not written, as soon as you write it, it does not go to the target.

So typically, it goes in batches. So, you can imagine that like, for example, a delivery truck. So if you want to deliver stuff, what will happen is that you will first load up the truck and when the truck is full, it will go or if you have seen some of these buses waiting at a bus terminus, so the conductor will keep waiting for passengers to fill up and only when all the passengers fill up does the bus leave.

So, the same way when you write output, only when the output has filled up to a certain length, which is called it. So, it is writes into what is a temporary space, which is usually called a buffer. So, this output is buffered. So, when you say print, for example, it does not automatically print on the screen.

In fact, if you see, if you monitor a long running program, and you have written some diagnostic print statements, and you do not see them on the screen, it does not necessarily mean that that print statement has not been executed, it just means that it has not passed the buffer and come onto the screen. So, the way to push things out of the buffer forcibly is to flush them.

So, if you are if you really want the information that is in the buffer to go out immediately, then you can call flush. So, close and flush are two operations which are not directly related to putting out specific data but to making sure that you have released the resources, or you have emptied out the buffer of pending writes. So, this is as far as reading and writing raw bytes are concerned.

(Refer Slide Time: 08:23)

IIT Madras
BSc Degree

Connecting a stream to an external source

- Input and output streams ultimately connect to external resources
 - A file, an internet connection, memory
 - ...
 - We limit ourselves to files
- Create an input stream attached to a file

```
var in = new FileInputStream("input.class");
```

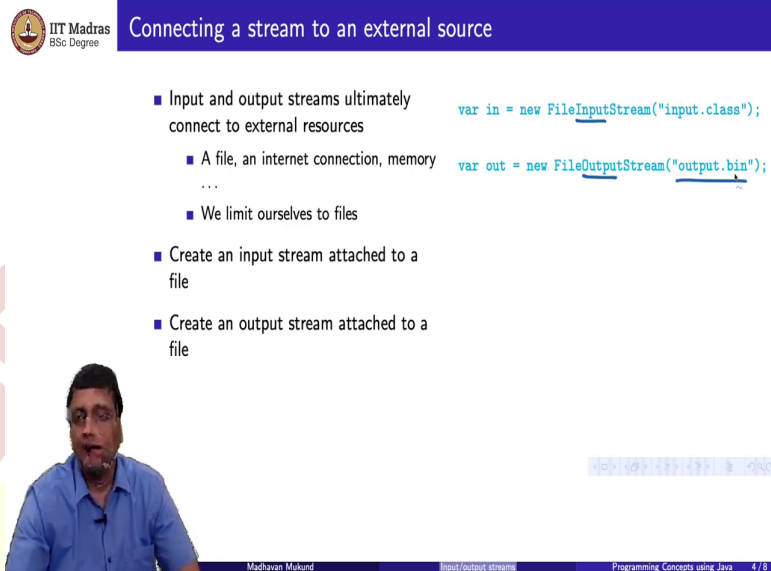
Madhavan Makund | Input/output streams | Programming Concepts using Java | 4 / 6

So, the first thing now we want to do is to make sure that our stream is connected to an external source, because we really do not have any input and output within the program, we have to talk to some external thing. And this external thing could be a file as we said, or an internet connection or even someplace in memory. So, in this class, we will just limit ourselves to dealing with files.

So, here is how you would construct an input stream attached to a file. So, you pass as a string, the name of the file. In this case, it is just a simple name, but you could give a path where it could find it on the disk on which your computer, your program is running. And then you ask for a new file input stream which opens to that file. So basically, henceforth, this

input will take input from that file and of course, it will be assigned to this variable which we have called in.

(Refer Slide Time: 09:16)



IIT Madras
BSc Degree

Connecting a stream to an external source

- Input and output streams ultimately connect to external resources
 - A file, an internet connection, memory ...
 - We limit ourselves to files
- Create an input stream attached to a file
- Create an output stream attached to a file

```
var in = new FileInputStream("input.class");  
var out = new FileOutputStream("output.bin");
```

Madhavan Mukund | Input/output streams | Programming Concepts using Java | 4/8

So symmetrically, you can create a stream which outputs to a file. So, as you would expect the input is replaced by output. So, you get a new file output stream. And once again, you provide a file name. So, now anything that I write to the stream out will actually go into that file, remember there is buffering things so not go immediately it will go in chunks. But, everything that is written to the stream out will actually land up in that file because this is an output stream connected to that file.

Now input is clear. When I read input from a file, the file is there I read I start from the beginning and I go to the end, but output there is an ambiguity. What if the file already exists? Supposing there is already a file called output dot bin and I start opening it for writing in this case. What does it mean? Does it mean that I am going to replace the current thing? Or I am going to add that is append to the current file. So, both are possible.

(Refer Slide Time: 10:12)



IIT Madras
BSc Degree

Connecting a stream to an external source

- Input and output streams ultimately connect to external resources
 - A file, an internet connection, memory ...
 - We limit ourselves to files
- Create an input stream attached to a file
- Create an output stream attached to a file
- Overwrite or append?
 - Pass a boolean second argument to the constructor

```
var in = new FileInputStream("input.class");  
  
var out = new FileOutputStream("output.bin");  
  
var out = new  
FileOutputStream("newoutput.bin", false);  
// Overwrite  
  
var out = new  
FileOutputStream("sameoutput.bin", true);  
// Append
```



Madhavan Mukund

Input/output streams

Programming Concepts using Java 4/8

So, when you create this output stream, there is an optional second argument. So there is a different constructor. So, the default is that you replace. So, the second constructor essentially tells second argument tells you whether you want to append or not. So, this is really answering the question. Is it an append? If append is false, it means I am going to overwrite. So, if I do not say anything, or if I write false, then this will create a new output file.

I have indicated by giving it a new name. But even if there was a file called new output dot bin, that file will get overwritten. On the other hand, if I say true, then it means that I do want to append. So, if I already have a file called same output dot bin, it will start writing at the end of the file. But it is not going to insist that the file exists, obviously. So, if there is no file, it will create a new one. It is just that if the file exists, then it will append so you can control whether you are overwriting or appending, when you open an output file string.

सिद्धिर्भवति कर्मजा

(Refer Slide Time: 11:10)



Reading and writing text

- Recall **Scanner** class
- Can apply to any input stream

```
var fin = new FileInputStream("input.txt");  
var scin = new Scanner(fin);  
  
var scin = new Scanner(  
    new FileInputStream("input.txt")  
);
```



Reading and writing text

- Recall **Scanner** class
- Can apply to any input stream
- Many read methods

```
var fin = new FileInputStream("input.txt");  
var scin = new Scanner(fin);  
  
var scin = new Scanner(  
    new FileInputStream("input.txt")  
);  
  
String s = scin.nextLine(); // One line  
String w = scin.next(); // One word  
int i = scin.nextInt(); // Read an int  
boolean b = scin.hasNext(); // Any more words?
```



So now, we have seen how to abstractly read and write bytes, which are uninterpreted, and how to connect a stream to a file. So, you could take that file stream and apply the read and write function, and you will read and write bytes from a file. But normally, reading and writing bytes from a file is not terribly useful. What you want to do is interpret those bytes in terms of the data that your program manipulates. So, one of the common things that you want to do is to read and write text files. So, let us see how to read and write text files.

So, in the very first week of the second week of the course, when we discuss Java, for the first time, we talked about basic input and output in Java. And we talked about the console class and a scanner class. And we talked about attaching scanner to system input stream,

which is essentially your keyboard called system dot in. So, this scanner mechanism actually works for any input stream.

So, recall that we could have created a file input stream connected to this file. So, we can now take that input stream. And in some sense, pipe it to a scanner. So, this is taking a stream which is connected to a file and connecting that stream to a scanner. So, you can also do it directly, you can start a new scanner in which as an argument to directly create the file input stream without naming it. So, once we have done that, then this scanner input stream, we have already seen it in a little detail at the beginning. But, we can see it in more detail.

As, a number of methods to read text data from the input stream in this case is connected to a file. But logically, if we had connected it to some other device, like a website or something like that, it will be reading from there. So, the scanner does not care where the data is coming from, it is coming to it in a stream from somewhere, and that somewhere was supplied to it as a string.

So, now if I read next line, for instance, then this is text data. So, it will read one line up to a newline character and store it as a string and return it. If I just say next, it will give me the next word. So, next word is defined by default be delimiting it by a space or a newline or any whitespace. Actually, you can change the delimiter if you want, but we will not get into the details.

Or you could read the next integer which will directly return with path and return an integer. And if you remember how input works in Python, you only read strings, and then you have to explicitly invoke an int function to convert it. So, here in some sense is reading a string, converting it to an integer and then storing it as an integer.

And then you can also check whether there are any more words. So, this returns a Boolean, true if there is a word still to come, it is like an iterator, remember, this has next. So, is there one more word to read or not? So, once you have a scanner class, then an object of scanner class can invoke on its input stream a variety of methods to read text data in different formats, either as a string, or as an integer and so on.

(Refer Slide Time: 14:11)



Reading and writing text

- Recall **Scanner** class
 - Can apply to any input stream
- Many read methods
- To write text, use **PrintWriter** class
 - Apply to any output stream

```
var fout = new FileOutputStream("output.txt");
var pout = new PrintWriter(fout);

pout var = new PrintWriter(
    new FileOutputStream("output.txt");
);
```



Reading and writing text

- Recall **Scanner** class
 - Can apply to any input stream
- Many read methods
- To write text, use **PrintWriter** class
 - Apply to any output stream
- Use **println()**, **print()** to write txt

```
var fout = new FileOutputStream("output.txt");
var pout = new PrintWriter(fout);

pout var = new PrintWriter(
    new FileOutputStream("output.txt");
);

String msg = "Hello, world";
pout.println(msg);
```

System.out.println()

So symmetrically, the simplest way to write text data is to create what is called a **PrintWriter**. So again, you have a stream already given to you. Usually an output stream attached to a text file in this case, and on top of that, you build a **PrintWriter**. And as before, you can create the **PrintWriter** directly by passing the stream constructor as the argument to it, you do not have to separately create the output stream and then pass it to the **PrintWriter**. And just like the **Scanner** class had various methods to read.

The **PrintWriter** has different methods to print but these are the methods that we already know. So remember, we you we have been writing all along **system dot out**, **dot println**. So, all of those functions that we associated with **system dot out**, are actually the same functions

which are available to `PrintWriter`. So, I can take a string and I can write to this, this `PrintWriter` stream that I created, I can say `p out dot print ln message`.

So, we will print it and put a newline character and so on. So, we have these read methods and these print methods.

(Refer Slide Time: 15:20)

Reading and writing text

- Recall `Scanner` class
 - Can apply to any input stream
- Many read methods
- To write text, use `PrintWriter` class
 - Apply to any output stream
- Use `println()`, `print()` to write txt
- Example: Copy input text file to output text file

```
var in = new Scanner(...);
var out = new PrintWriter(...);

while (in.hasNext()){
    String line = in.nextLine();
    out.println(line);
}
```

Madhavan Mukund | Input/output streams | Programming Concepts using Java | 5/8

So, let us put that together. So, supposing I want to take a text file and make a copy of it. So say, I have some input dot text, and I want to make an output dot text, then from the previous discussion, you will first create a scanner object out of the file input stream connected to the input file, then you will create a `PrintWriter` object. So, this `PrintWriter`, we will write a `PrintWriter` object out of the output stream connected to the output file out file dot text or whatever.

And now you can use this `hasNext` to keep reading lines, as long as the input file has lines. So, so long as the input file has something to read, you read the next line, and you print it out to the output, so I am just copying from input output. So, it is a very simple program, but just illustrates how you would use this input and output mechanism in order to read texts and write text.

(Refer Slide Time: 16:16)



IIT Madras
BSc Degree

Reading and writing text

- Recall `Scanner` class
 - Can apply to any input stream
- Many read methods
- To write text, use `PrintWriter` class
 - Apply to any output stream
- Use `println()`, `print()` to write txt
- Example: Copy input text file to output text file
- Beware: input/output methods generate many different kinds of exceptions
 - Need to wrap code with `try` blocks

```
var in = new Scanner(...);  
var out = new PrintWriter(...);  
  
try  
{  
    while (in.hasNext()){  
        String line = in.nextLine();  
        out.println(line);  
    }  
}
```



Madhavan Mukund

Input/output streams

Programming Concepts using Java

5 / 8

Now, this program is deceptively, deceptively simple, because actually, all these methods which are available in this IO stream, business in Java carries a burden of throwing a lot of exceptions, because it is very common in such situations to have exceptions, because the file that you try to open may not be found and you are trying to write something the disk may be full, and so on.

So, every one of these things will actually end up having to be inside it, including opening a file. So, when you write this, it will typically if you just write an, if function to do this, Java will initially complain about various exceptions that you are not handling. So, you will have to look up the documentation to figure out what is the most optimal way to write this code with the appropriate try blocks around it. So, this is as far as reading and writing text. So, remember, we had this file input stream file output stream, and we wrapped a scanner and a `PrintWriter` around them.

(Refer Slide Time: 17:12)



IIT Madras
BSc Degree

Reading and writing binary data

- To read binary data, use `DataInputStream` class
 - Can apply to any input stream

```
var fin = new FileInputStream("input.class");  
var din = new DataInputStream(fin);  
  
var din = new DataInputStream(  
    new FileInputStream("input.class")  
);
```



Madhavan Mukund

Input/output streams

Programming Concepts using Java

6 / 8

So, what if instead you wanted to read it as binary data, supposing we have actually written 4 byte integers as raw data onto a file, I want to read and write such data. So, here instead of scanner, we use something called a data input stream. So, it is a different class. But the mechanism of setting it up is exactly the same a scanner, you start by creating a file input stream, and then you wrap a data input stream around it.

So, this is the way in which Java has modularize this process, so you have the same process for connecting to a file. And now you take the output from the input from that file and whether you want to read it as text, use a scanner, if you want read binary data use a data input stream.

(Refer Slide Time: 17:56)



Reading and writing binary data

- To read binary data, use `DataInputStream` class
 - Can apply to any input stream
- Many read methods

```
var fin = new FileInputStream("input.class");
var din = new DataInputStream(fin);
```

```
var din = new DataInputStream(
    new FileInputStream("input.class")
);
```

```
readInt, readShort, readLong
readFloat, readDouble,
readChar, readUTF
readBoolean
```



Madhavan Makund

Input/output streams

Programming Concepts using Java 6/8

And since this is a data input stream, it will allow you to read raw data and the different formats that are available in the scalar data types of Java. So, you can read an integer, you can read a short, you can read a long, you can read a float, you can read a double line and so on. So, instead of just reading text, as you would, in a scanner class, here, you will read raw data and put it into the appropriate data type in one of the built in classes built in data types of Java.

(Refer Slide Time: 18:23)



Reading and writing binary data

- To read binary data, use `DataInputStream` class
 - Can apply to any input stream
- Many read methods
- To write binary data, use `DataOutputStream` class
 - Apply to any output stream

```
var fout = new FileOutputStream("output.bin");
var dout = new DataOutputStream(fout);
```

```
var dout = new DataOutputStream(
    new FileOutputStream("output.bin")
);
```



Madhavan Makund

Input/output streams

Programming Concepts using Java 6/8

So again, symmetrically, instead of the `PrintWriter` class, you have a data output stream class, so you again, create a file output stream, and wrap a data output stream around it just like earlier, you created a file output stream and wrapped a `PrintWriter` around it.

(Refer Slide Time: 18:39)



IIT Madras
BSc Degree

Reading and writing binary data

- To read binary data, use `DataInputStream` class
 - Can apply to any input stream
- Many read methods
- To write binary data, use `DataOutputStream` class
 - Apply to any output stream
- Many write methods

```
var fout = new FileOutputStream("output.bin");
var dout = new DataOutputStream(fout);

var dout = new DataOutputStream(
    new FileOutputStream("output.bin")
);

writeInt, writeShort, writeLong
writeFloat, writeDouble
writeChar, writeUTF
writeBoolean
writeChars
writeByte
```



Madhavan Mukund Input/output streams Programming Concepts using Java 6/8

And once again, just like you had various methods to read raw data, you have various methods to write raw data. So, you can write in short, you can write characters, you can write byte, you can write Boolean. So, you need to look up the documentation to figure out exactly what some of these mean, like write UTF, what does it mean and so on. So, just like we wrote a small loop to read an input text file and write it to an output text file.

(Refer Slide Time: 19:07)



IIT Madras
BSc Degree

Reading and writing binary data

- To read binary data, use `DataInputStream` class
 - Can apply to any input stream
- Many read methods
- To write binary data, use `DataOutputStream` class
 - Apply to any output stream
- Many write methods
- Example: Copy input binary file to output binary file
 - Again, be careful to catch exceptions

```
var in = new DataInputStream(...);
var out = new DataOutputStream(...);

int bytesAvailable = in.available();
while (bytesAvailable > 0){
    var data = new byte[bytesAvailable];
    in.read(data);
    out.write(data);
    bytesAvailable = in.available();
}
```



Madhavan Mukund Input/output streams Programming Concepts using Java 6/8

You can do the same thing to copy a binary file. So again, you would wrap a data input stream around a file input stream, you would wrap a data output stream around a file output stream. So, you would set up this in and out. And now we go back to that thing we mentioned

earlier, the raw byte input. So here, we are just copying. So, we do not want to interpret we do not care what, we just read a byte write a byte.

So, we check first of all, how many bytes are available to read. And while this is greater than 0, we read all the bytes that are available into a data array. So, we read it here, we write it here. And then we again check how many bytes are available. We keep going until the number of bytes available is 0.

So, it may happen that it all happens in one chunk. It may happen it happens in multiple chunks, we do not care. As long as it keeps going until the says no bytes are available. You are going to faithfully copy all the input bytes to the output bytes. Since you are reading and writing them as bytes, you are not really interpreting them, you do not care, you could do the same thing in terms of integers and floats, and so on. But here, we are just doing a raw binary code.

(Refer Slide Time: 20:13)

IIT Madras
BSc Degree

Other features

- Buffering an input stream
 - Reads blocks of data
 - More efficient

```
var din = new DataInputStream(  
    new BufferedInputStream(  
        new FileInputStream("grades.dat")  
    )  
);
```

~ [Buffer] →

Madhavan Makund | Input/output streams | Programming Concepts using Java | 7/8

- Buffering an input stream
 - Reads blocks of data
 - More efficient

```
var din = new DataInputStream(  
    new BufferedInputStream(  
        new FileInputStream("grades.dat")  
    )  
);
```



Madhavan Makund

Input/output streams

Programming Concepts using Java 7/8

So, these are two of the basic things that one can do with streams, connect them to a file or some other output source, and then read the data as text or read the data as some binary format, which is meaningful for the built in data types. But there are many other aspects of streams, which we can consider.

So, the first thing is that we remember we talked about output streams being buffered. So, an output stream is buffered means that first when I write something, it fills up this thing, and only then it gets written. So, there is a lag between filling up the buffer. And so this is the bus getting full, and then it takes off and the bus is full. So, the same thing can be done on the input direction.

Instead of reading one thing at a time. So, imagine that you are in a supermarket and the shelf becomes empty. So, somebody comes and asks you for toothpaste, and the shelf is empty. Now you could go into the store room at the back and bring one toothpaste, but you would not do that you would typically bring out a bunch of toothpaste.

So, you will bring out a box of toothpaste because then you know that later on more people are going to come and ask for toothpaste. So, you will save yourself multiple times by doing it in one shot. So, same way you can read data also in chunks. So, if I only want to read one character, eventually, I can assume that I am going to read more characters. So, I can read a chunk of data because when I read there is a certain size of data that is automatically available to me.

But how do you indicate that such a facility is there in Java. Well, you can create explicitly a buffered input stream. So by default, a file input stream is not buffered. But now if I create a buffered input stream around it, then the data is going to be read more efficiently. That is all it is not going to change the data, it is just going to read it more efficiently.

And I can now take this combination. So, I pipelined it, I have taken an file input stream, put it into a buffered input stream. So now, the data from here is being buffered. And then I feed it to a data input stream so that I can interpret the data as binary data. So, this is the kind of thing that can do with buffers.

(Refer Slide Time: 22:17)

Other features

- Buffering an input stream
 - Reads blocks of data
 - More efficient
- Speculative reads
 - Examine the first element
 - Return to stream if necessary

```
var din = new DataInputStream(  
    new BufferedInputStream(  
        new FileInputStream("grades.dat")  
    )  
);  
  
var pbin = new PushbackInputStream(  
    new BufferedInputStream(  
        new FileInputStream("grades.dat"))) );  
  
int b = pbin.read();  
if (b != '<') pbin.unread(b);
```

Madhavan Mukund | Input/output streams | Programming Concepts using Java | 7/8

Another thing that you can do is to examine something which is available in the stream without consuming it. So, when we looked at a long time ago, and collections, we looked at these peek methods, when we could look at the top of the stack or the front of the queue and then not read it actually. So, the same thing we can do with what is called a pushback stream.

So, we can take this buffered stream, and then we can further layer it as a pushback stream. So now, in this pushback stream, I can read something and then check its value. And if I do not mean depending on whatever the value is, I can either consume it. So, if I read it, it has been consumed or I can unread it, put it back, hence push back so I can read it, look at it and then put it back.

So, one thing to remember is that these things are designed for minimal functionality. So, file input stream can only connect to a file, it cannot read anything. Scanner class can read text,

but it cannot connect to a file. So, each of these streams that we have discussed does one thing or one type of thing and not many types of things.

(Refer Slide Time: 23:33)



Other features

- Buffering an input stream
 - Reads blocks of data
 - More efficient
- Speculative reads
 - Examine the first element
 - Return to stream if necessary
- Streams are specialized
 - `PushBackStream` can only `read()` and `unread()`
 - Feed to a `DataInputStream` to read meaningful data

```
var pbin = new PushbackInputStream(  
    new BufferedInputStream(  
        new FileInputStream("grades.dat")));  
  
var din = new DataInputStream(pbin);
```



Madhavan Makund Input/output streams Programming Concepts using Java 7/8

So, here a push back stream can only read and push back read it cannot do any anything else other than read and unread. So, if I then want to take this data that I have read and unread and interpret it as a binary data, then I must take this stream and create a data input stream on top of it. So, before I invoke the data input stream, I use the pushback stream to check whether the data should be passed to the data input stream or not. So, if I unread it then it goes and if I read it the data input stream will not see it. So in this way, Java separates the concerns.

(Refer Slide Time: 24:10)



Other features

- Buffering an input stream
 - Reads blocks of data
 - More efficient
- Speculative reads
 - Examine the first element
 - Return to stream if necessary
- Streams are specialized
 - `PushBackStream` can only `read()` and `unread()`
 - Feed to a `DataInputStream` to read meaningful data

```
var pbin = new PushbackInputStream(  
    new BufferedInputStream(  
        new FileInputStream("grades.dat")));  
  
var din = new DataInputStream(pbin);
```

- Java has a whole zoo of streams for different tasks
 - Random access files, zipped data, ...



Madhavan Makund Input/output streams Programming Concepts using Java 7/8

- Buffering an input stream
 - Reads blocks of data
 - More efficient
- Speculative reads
 - Examine the first element
 - Return to stream if necessary
- Streams are specialized
 - `PushBackStream` can only `read()` and `unread()`
 - Feed to a `DataInputStream` to read meaningful data

```
var pbin = new PushbackInputStream(
    new BufferedInputStream(
        new FileInputStream("grades.dat")));

var din = new DataInputStream(pbin);
```

- Java has a whole zoo of streams for different tasks
 - Random access files, zipped data, ...
- Chain together streams in a pipeline
 - Read binary data from a zipped file

```
FileInputStream →
ZipInputStream →
DataInputStream
```



Madhavan Mukund

Input/output streams

Programming Concepts using Java 7/8

So, so there is a whole zoo of streams. So, we have looked already we have seen some five or six different types of streams in this small lecture. And there is actually a whole collection of streams to do deal with a variety of tasks. For example, files, the files we have been talking about are what are called sequential files. I read a start at the beginning and I go to the end. So, I cannot vary where I am going I have to start at the back and go to the end.

Similarly, when I write as at the beginning go to the end, so what if I want to read from the middle of a file or write at some place, not at the end not at end but write somewhere in the middle of the file. So, these are what are called random access files, I might want to say at the 1000 byte got write something or read from the 700 byte onwards. So, Java has a separate random access file stream class which allows you to do that.

What if the data is somehow compressed, supposing it is in some zip format, then I have to unzip it in order to make sense of it. So, rather than unzip it separately, Java will actually allow you to read the zip data directly, and transforms zip data into unzip data.

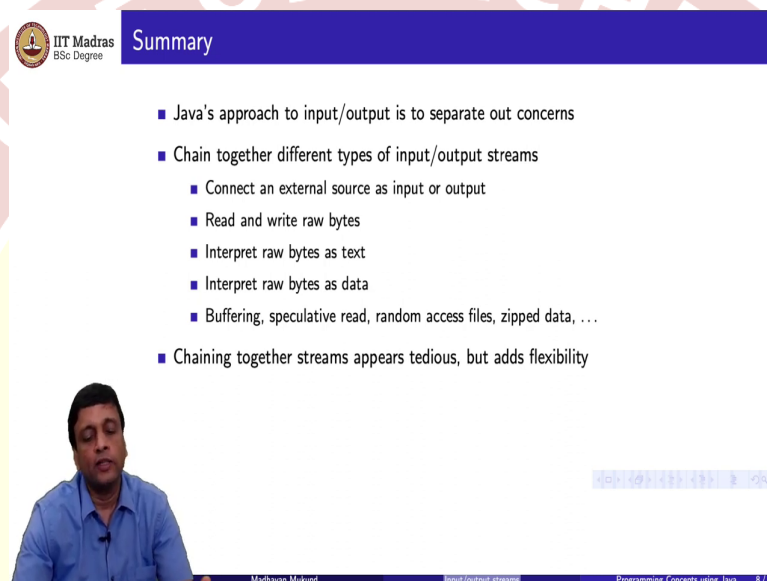
So, for example, if I want to read binary data from a zip file, so first of all, I will construct a file input stream connected to the zip file, then I will layer a zip input stream on top of this, which will take the data from that file input stream and unzip it and put it out as unzip data.

Now, I can take the unzip data, which is now readable, and interpret it using a data input stream. So, by combining these components, you can put together these complex

combinations using a relatively small number of objects. So, the number of permutations and combinations of these objects is very large.

So, rather than confronting you with a zoo of high level functions, which combine all these functionalities, the option that Java has chosen is to give you a small, smaller set, not small, but a smaller set of low level things, each of which does a specialized thing, and leave it to you to assemble them as you would see fit.

(Refer Slide Time: 26:10)



IIT Madras
BSc Degree

Summary

- Java's approach to input/output is to separate out concerns
- Chain together different types of input/output streams
 - Connect an external source as input or output
 - Read and write raw bytes
 - Interpret raw bytes as text
 - Interpret raw bytes as data
 - Buffering, speculative read, random access files, zipped data, ...
- Chaining together streams appears tedious, but adds flexibility

Madhavan Makund Input/output streams Programming Concepts using Java 9/8

So, the basic approach that Java has taken to input an output is to separate out the concerns. So, you are supposed to take these different input output streams and chain them together according to your requirement. So, at the basic level, you want to connect it to some different external source for input or output. And, it does not really matter once you have connected it, for the rest of the processing, whether it came from a file, or it came from a network device, or it came from a memory or something else.

Then, at a raw level, these things transmit bytes, but you can take these bytes and interpret the mistakes, you can interrupt them as data. They could be zipped. You could buffer it, you could check an unread, you could do speculative access at the file level, you could do random access. So, by doing all this, at one level, it seems that could do anything basic. If I just want to read a text file, it seems so much simpler if I usually this gave me a single function, which will open a text file and read it like in Python, which is open a file, read it and close it.

Whereas, here we have to set up the file stream, then we have to set up a scanner class and then we have to call functions on them. But on the other hand, by doing it this way, though, it appears tedious. You have this flexibility, you can combine different sources with different types of data interpretation very flexibly. So, it is a trade off. But in the long run, having these different parts which you can put together as you see fit gives you a much wider range of options than if you are just given a fixed set of functions, which work with fixed types of inputs and fixed types of data.

